Real-Time Object Detection Using YOLOv8: A High-Efficiency Deep Learning Approach

V. Grégoire--Bégranger (SME), M. Jalama (ELSS)
Institut Polytechnique des Sciences Avancées de Paris

Object detection is a key task in computer vision with applications in autonomous systems, surveillance, and robotics. This project presents a real-time object detection application using the YOLOv8 deep learning model with python. Using libraries like OpenCV, the system identifies and tracks multiple object types in a video. Thanks to a pre-trained model based on the COCO dataset, the application supports class selection and bounding box visualization. This study demonstrates the ease of implementation of a computer vision algorithm using common libraries.

Introduction

In recent years, the field of computer vision has witnessed advancements, particularly in the domain of real-time object detection and recognition. An illustration of the capabilities of modern image recognition systems is found in Tesla's autonomous driving technologies. Tesla vehicles rely heavily on neural networks capable of interpreting live video feeds from multiple cameras to detect and classify objects in dynamic environments. These algorithms, optimized for both speed and accuracy, are enabling autonomous navigation, showcasing the transformative potential of real-time visual perception in embedded systems.

The growing need for machines to understand and interact with their environment has positioned object detection as a cornerstone of artificial intelligence applications, from surveillance and robotics to medical imaging and intelligent transportation systems. Among the state-of-the-art models developed for this purpose, the *You Only Look Once* (YOLO) family of algorithms stands out for its ability to perform object detection with both high accuracy and exceptional speed.

This project presents a real-time object detection application using YOLOv8 with Python. The system uses a pre-trained model on the COCO dataset to detect common object categories in images or live video. The application integrates OpenCV for image processing and `tkinter` for a basic graphical interface. Users can select classes to display, and view bounding boxes across frames.

Our goal is to replicate, at a smaller scale, the ability to detect, and track multiple objects in video streams. The project illustrates how deep learning models can be integrated into user-facing tools using common Python libraries. Additionally, it highlights the practical use of pre-trained models for rapid development without requiring access to large datasets or high-performance computing resources.

YOLOv8 Object Detection Algorithm

YOLOv8 is a single-stage object detector based on a convolutional neural network. It consists of a backbone for feature extraction, a neck to aggregate features at different scales, and a detection head that outputs bounding boxes, objectness scores, and class probabilities.

The model processes the input image in one forward pass. For each grid cell, it predicts bounding boxes along with confidence and class scores. Post-processing includes applying a confidence threshold and non-maximum suppression (NMS) to remove duplicate detections.

YOLOv8 models are available in various sizes, trading off speed and accuracy. This project uses a pretrained model trained on the COCO dataset, which contains over 200k labeled images across 80 object classes. The dataset supports a wide range of everyday objects, making it suitable for general-purpose detection.

System Implementation

The application is written in Python. It uses the Ultralytics `YOLOv8` package for model inference, OpenCV for image processing, and `tkinter` for the GUI. The model used is YOLOv8n pretrained on the COCO dataset.

The program captures video frames using OpenCV, resizes them, and passes them to the YOLOv8 model. Detections are filtered by confidence score and visualized with bounding boxes and class labels.

Application Functionalities Overview

The application provides a graphical user interface for real-time object detection in video streams. Developed with the `tkinter` library, it supports two video input sources:

- A predefined set of sample videos included with the application
- User-selected video files from the local file system

These options facilitate both demonstration and testing on custom data.

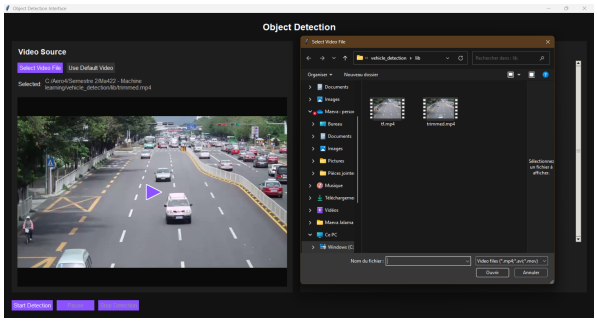

Figure 1: *User interface for video source selection*

Users can select object categories for detection from a list based on the pre-trained model's classes (e.g., "person," "car," "bus," "umbrella"). Each selected class is assigned a distinct color to differentiate detected objects visually in the output.

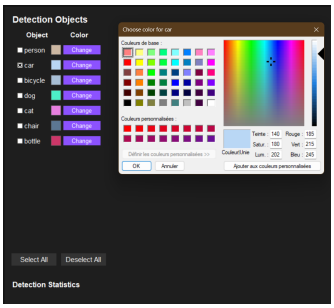

Figure 2: *Selection of object classes for detection*

During detection, the video stream is displayed with bounding boxes drawn around identified objects. Bounding box colors correspond to the selected classes. The interface includes controls to start, pause, and stop the detection process.

Figure 3: *Video stream with detected objects*

1 Detailed Code Implementation

The project includes three main Python scripts that demonstrate different modes of interaction with the YOLOv8 object detection model: a command-line interface (CLI), a threaded backend module for real-time processing, and a graphical interface built with

Tkinter. Each script builds upon the same core detection logic but adapts it to different application scenarios.

Command-Line Implementation

The script `non_threading_CLI_YOLO.py` provides a minimal, non-threaded implementation of YOLOv8-based object detection. It uses OpenCV to capture and display frames from a video file, and the Ultralytics YOLOv8 API to run inference on every other frame for efficiency.

Bounding boxes and class labels are drawn directly onto the video frames, and a randomly assigned color is used for each class. This version is suitable for testing detection accuracy or quickly validating the behavior of the model without user interface complexity.

Threaded Detection Module

The script `ObjectDetector.py` encapsulates the object detection pipeline within a Python class, designed to run inference in a background thread. This module provides start, pause, resume, and stop functionalities, allowing the detection process to operate independently of the main application thread.

This class is central to integrating object detection into interactive applications, ensuring that real-time video analysis does not block user interactions. The detections are filtered based on a user-defined subset of COCO classes, and results are drawn on the video frames using OpenCV.

Key features include:

- Thread-safe detection and frame storage
- Frame-level result tracking with bounding boxes
- Live updating of current frame for GUI display

Graphical User Interface

The script `app.py` implements a full-featured graphical interface using the Tkinter library. It integrates the `ObjectDetector` class to provide responsive real-time detection with a user-friendly layout.

The interface includes:

- Video source selection (user file or default sample)
- Class selection with custom color assignment
- Real-time preview with bounding boxes
- Detection control (start, pause, stop)

The GUI employs a dark theme for visual clarity and allows users to configure detection parameters before launching the threaded detection process. It also supports rescaling and video preview rendering using the Pillow library. The combination of GUI and background detection thread ensures a smooth user experience without performance bottlenecks.

Overall, the codebase is structured to separate detection logic from interface logic, making the application modular and maintainable. The use of pre-trained YOLOv8 weights and COCO class labels enables immediate deployment without model training or complex preprocessing.

Conclusion

This project demonstrates the effective integration of the YOLOv8 deep learning model into a real-time object detection system using Python. By using pre-trained weights and standard computer vision libraries such as OpenCV and Tkinter, the application achieves accurate and efficient detection with an accessible user interface. The modular design separating detection and interface components supports flexibility and potential future enhancements. Overall, this work highlights the practical deployment of state-of-the-art object detection methods for real-time applications without the need for extensive dataset preparation or specialized hardware.